

Guía Pedagógica: Día 6 - Navegación y Rutas con Parámetros

Hoy daremos un salto gigante: transformaremos nuestra aplicación de una sola página estática en una experiencia dinámica con múltiples vistas. Aprenderemos cómo configurar el **Angular Router**, cómo pasar información entre páginas mediante la URL y cómo usar la característica más moderna de Angular para recibir parámetros: el **Component Input Binding**.

Contexto Pedagógico: ¿Qué es el Routing?

En las aplicaciones modernas (SPA - Single Page Applications), no cambiamos de archivo HTML al navegar. En su lugar, el **Router** de Angular observa la URL del navegador y, dependiendo de lo que vea (ej. `/movie/123`), decide qué componente debe "dibujar" en la pantalla.

El Reto de hoy:

Queremos que al hacer clic en una película, el navegador nos lleve a `/movie/[ID_DE_LA_PELICULA]` y que esa nueva página sepa exactamente qué película debe cargar.

Paso 1: Configurando el "Cerebro" de la Navegación

Primero, debemos decirle a Angular que existe una nueva ruta que acepta un parámetro variable (el ID).

1. Activar el Enlace de Parámetros

Abre `src/app/app.config.ts`. Vamos a añadir una configuración especial llamada `withComponentInputBinding()`. Esto permite que los parámetros de la URL lleguen a nuestros componentes como si fueran simples `@Input()`.

```
// src/app/app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    // ... otros providers
    provideRouter(routes, withComponentInputBinding()),
    // ...
  ],
};
```

2. Definir la Ruta con Parámetro

Abre **src/app/app.routes.ts**. Añadiremos la ruta para los detalles. El uso de `:id` indica que esa parte de la URL es un parámetro variable.

```
// src/app/app.routes.ts
export const routes: Routes = [
  {
    path: '',
    loadComponent: () => import('./features/home/home').then(m =>
m.Home)
  },
  {
    path: 'movie/:id', // :id es el parámetro dinámico
    loadComponent: () => import('./features/movie-details/movie-
details').then(m => m.MovieDetails)
  }
];
```

Paso 2: El Servicio - Buscando una Película Específica

Nuestro `MovieService` necesita un nuevo método para pedirle a TMDB los detalles de una sola película usando su ID.

Abre **src/app/core/services/movie.service.ts**:

```
getMovieById(id: string | number) {
  return this.http.get<Movie>(`${this.apiUrl}/movie/${id}`);
}
```

Paso 3: El Componente de Detalles (MovieDetails)

Este componente es el que se mostrará cuando naveguemos a `/movie/:id`.

1. La Lógica (TypeScript)

Nota lo sencillo que es recibir el ID gracias a la configuración que hicimos en el Paso 1.

```

@Component({
  selector: 'app-movie-details',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './movie-details.html',
  styleUrls: ['./movie-details.css']
})
export class MovieDetails implements OnInit {
  private movieService = inject(MovieService);

  // Angular Llenará esto automáticamente con el valor de :id de la URL
  @Input() id!: string;

  movie = signal<Movie | null>(null);

  ngOnInit(): void {
    this.movieService.getMovieById(this.id).subscribe({
      next: (movie) => this.movie.set(movie)
    });
  }
}

```

2. La Vista (HTML)

Usamos el **Control Flow** @if para mostrar un estado de carga mientras recibimos los datos.

```

@if (movie(); as movieData) {
  <div class="details-container">
    <!-- Imagen de fondo (Backdrop) -->
    <div class="backdrop-wrapper">
      <img [src]="`https://image.tmdb.org/t/p/original` +
movieData.backdrop_path" class="backdrop-image">
      <div class="backdrop-overlay"></div>
    </div>

    <div class="content-wrapper">
      <h1 class="movie-title">{{ movieData.title }}</h1>
      <p class="overview">{{ movieData.overview }}</p>

      <button class="back-button" onclick="history.back()">←
Volver</button>
    </div>
  </div>
} @else {
  <div class="loading">Cargando detalles...</div>
}

```

Paso 4: Activando el clic en la Tarjeta (MovieCard)

Para que el usuario pueda navegar, debemos vincular nuestras tarjetas con la nueva ruta.

1. Importar el RouterModule

En movie-card.ts, añade RouterModule a la lista de imports.

2. Añadir el routerLink

En movie-card.html, envolvemos nuestra tarjeta.

```
<!-- src/app/shared/components/movie-card/movie-card.html -->
<div class="movie-card" [routerLink]="['/movie', movie.id]">
  <!-- ... resto del contenido ... -->
</div>
```

***Tip:** Usamos una sintaxis de array ['/ruta', parametro] para que Angular construya la URL correctamente por nosotros.*

Paso 5: Configuración de Server-Side Rendering (SSR) para Producción

Si intentamos subir nuestra aplicación a **Vercel** o hacer un *build* de producción en este punto, obtendríamos un error. ¿Por qué?

Angular, por defecto, intenta hacer **Pre-rendering** (generar archivos HTML estáticos durante el build) de todas las rutas. Sin embargo, para una ruta dinámica como movie/:id, Angular no sabe qué películas existen de antemano (¡hay miles en TMDb!), por lo que no puede generar un HTML estático para cada una.

Para solucionar esto, debemos decirle a Angular que la ruta de detalles debe generarse **bajo demanda** en el servidor (SSR) cuando el usuario la visita, no durante el build.

1. Configurar las rutas del servidor

Abre **src/app/app.routes.server.ts** y añade una regla específica para nuestra nueva ruta ANTES de la regla general (**).

```
// src/app/app.routes.server.ts
import { RenderMode, ServerRoute } from '@angular/ssr';

export const serverRoutes: ServerRoute[] = [
  {
    path: 'movie/:id',
    renderMode: RenderMode.Server, // SSR: Renderizar bajo demanda
    // en el servidor
  },
  {
    path: '**',
    renderMode: RenderMode.Prerender, // Pre-renderizar el resto
    // (como La Home)
  },
];
```

Con esto, nuestra página principal (/) seguirá siendo súper rápida porque se genera en el momento de compilar, mientras que los detalles de cada película (/movie/:id) se generarán en vivo cuando el usuario navegue hacia ellos.

Prueba de Aprendizaje (Checklist)

1. **¿Qué significa el colon : en una ruta como movie/:id?** (Respuesta: Indica que esa parte de la URL es un parámetro dinámico que puede cambiar, como un ID o un nombre de usuario).
2. **¿Para qué sirve withComponentInputBinding() en la configuración del Router?** (Respuesta: Permite que los parámetros de la ruta (como :id) se inyecten directamente en el componente usando el decorador @Input(), sin necesidad de usar el servicio ActivatedRoute).
3. **¿Por qué usamos loadComponent con un import() dinámico en las rutas?** (Respuesta: Para implementar **Lazy Loading**. Esto hace que el código del componente de detalles solo se descargue cuando el usuario realmente navega hacia esa página, haciendo que la carga inicial de la App sea mucho más rápida).

Control de Versiones

Guarda tu progreso:

```
git add .
git commit -m "feat: implementar navegación, detalles y configurar
SSR"
git push origin main
```